



Date: \_\_ / \_\_ / \_\_

## Experiment 4

4. a. You are given an undirected graph represented by an adjacency matrix. Write a C Program to perform Breadth-First Search (BFS) traversal starting from a given vertex. The function should return the order in which the vertices are visited during the traversal.

### Algorithm

#### ALGORITHM $BFS(G)$

```
//Implements a breadth-first search traversal of a given graph
//Input: Graph  $G = \langle V, E \rangle$ 
//Output: Graph  $G$  with its vertices marked with consecutive integers
//      in the order they are visited by the BFS traversal
mark each vertex in  $V$  with 0 as a mark of being "unvisited"
count  $\leftarrow 0$ 
for each vertex  $v$  in  $V$  do
    if  $v$  is marked with 0
        bfs( $v$ )

bfs( $v$ )
//visits all the unvisited vertices connected to vertex  $v$ 
//by a path and numbers them in the order they are visited
//via global variable count
count  $\leftarrow$  count + 1; mark  $v$  with count and initialize a queue with  $v$ 
while the queue is not empty do
    for each vertex  $w$  in  $V$  adjacent to the front vertex do
        if  $w$  is marked with 0
            count  $\leftarrow$  count + 1; mark  $w$  with count
            add  $w$  to the queue
    remove the front vertex from the queue
```

Activat  
Go to Se

### Program

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 100

int queue[MAX_VERTICES];
int front = -1, rear = -1;

void enqueue(int data) {
```

Design and Analysis of Algorithms, Laboratory Manual

Department of Computer Science and Engineering, Dayananda Sagar University, Bengaluru,  
Karnataka.



```
if (rear == MAX_VERTICES - 1) {
    printf("Queue Overflow\n");
    return;
}
if (front == -1) {
    front = 0;
}
queue[++rear] = data;
}

int dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return -1;
    }
    return queue[front++];
}

int isEmpty() {
    return front == -1 || front > rear;
}

void bfs(int graph[MAX_VERTICES][MAX_VERTICES], int numVertices, int startVertex,
int visited[]) {
    visited[startVertex] = 1;
    enqueue(startVertex);

    while (!isEmpty()) {
        int currentVertex = dequeue();
        printf("%d ", currentVertex);

        for (int i = 0; i < numVertices; i++) {
            if (graph[currentVertex][i] == 1 && !visited[i]) {
                visited[i] = 1;
                enqueue(i);
            }
        }
    }
}

int main() {
    int numVertices, graph[MAX_VERTICES][MAX_VERTICES],
    visited[MAX_VERTICES];

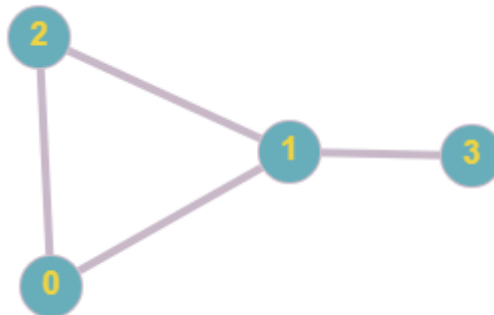
    printf("Enter the number of vertices: ");
    scanf("%d", &numVertices);

    printf("Enter the adjacency matrix:\n");
    for (int i = 0; i < numVertices; i++) {
```



```
for (int j = 0; j < numVertices; j++) {  
    scanf("%d", &graph[i][j]);  
}  
}  
  
printf("Enter the starting vertex for BFS: ");  
int startVertex;  
scanf("%d", &startVertex);  
  
for (int i = 0; i < numVertices; i++) {  
    visited[i] = 0;  
}  
  
printf("BFS Traversal: ");  
bfs(graph, numVertices, startVertex, visited);  
  
return 0;  
}
```

### Sample Output



```
Enter the number of vertices: 4  
Enter the adjacency matrix:  
0 1 1 0  
1 0 1 1  
1 1 0 0  
0 1 0 0  
Enter the starting vertex for BFS: 3  
BFS Traversal: 3 1 0 2
```



Enter the number of vertices: 4

Enter the adjacency matrix:

0 1 0 0

0 0 1 0

0 0 0 1

1 0 0 0

Enter the starting vertex for BFS: 0

BFS Traversal: 0 1 2 3